Assignment No -5

Name - Payal Balasaheb More . **Roll No** - EN23107080 . **Class** - TY(B) . **Batch** - A

Title - Business Model Development for AI Products Create a Business Model Canvas for an AI-based product. Define the value proposition, customer segments, revenue streams, and cost structures. Ethical Challenges in AI Analyze a real-world AI application (e.g., facial recognition, deep fake technology, AI bias in hiring) and discuss the ethical challenges involved. Propose solutions to mitigate risks and ensure responsible AI usage

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('/content/StudentPerformanceFactors.csv')
```

```
df
```

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Previous_Sc
0	23	84	Low	High	No	7	
1	19	64	Low	Medium	No	8	
2	24	98	Medium	Medium	Yes	7	
3	29	89	Low	Medium	Yes	8	
4	19	92	Medium	Medium	Yes	6	
...	...	...	...	...	...	...	
6602	25	69	High	Medium	No	7	
6603	23	76	High	Medium	No	8	
6604	20	90	Medium	Low	Yes	6	
6605	10	86	High	High	Yes	6	
6606	15	67	Medium	Low	Yes	9	

6607 rows × 20 columns

```
df.shape
```

```
(6607, 20)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6607 entries, 0 to 6606
Data columns (total 20 columns):
 #   Column                      Non-Null Count  Dtype 
---  ------                      --------------  ----- 
 0   Hours_Studied               6607 non-null   int64 
 1   Attendance                  6607 non-null   int64 
 2   Parental_Involvement        6607 non-null   object
 3   Access_to_Resources         6607 non-null   object
 4   Extracurricular_Activities  6607 non-null   object
 5   Sleep_Hours                 6607 non-null   int64 
 6   Previous_Scores             6607 non-null   int64 
 7   Motivation_Level            6607 non-null   object
 8   Internet_Access             6607 non-null   object
 9   Tutoring_Sessions           6607 non-null   int64 
 10  Family_Income               6607 non-null   object
 11  Teacher_Quality             6529 non-null   object
 12  School_Type                 6607 non-null   object
 13  Peer_Influence              6607 non-null   object
 14  Physical_Activity           6607 non-null   int64 
 15  Learning_Disabilities       6607 non-null   object
 16  Parental_Education_Level    6517 non-null   object
 17  Distance_from_Home          6540 non-null   object
```

```
 18  Gender                       6607 non-null   object
 19  Exam_Score                   6607 non-null   int64
dtypes: int64(7), object(13)
memory usage: 1.0+ MB
```

```
df.describe()
```

	Hours_Studied	Attendance	Sleep_Hours	Previous_Scores	Tutoring_Sessions	Physical_Activity	Exam_Score
count	6607.000000	6607.000000	6607.00000	6607.000000	6607.000000	6607.000000	6607.000000
mean	19.975329	79.977448	7.02906	75.070531	1.493719	2.967610	67.235659
std	5.990594	11.547475	1.46812	14.399784	1.230570	1.031231	3.890456
min	1.000000	60.000000	4.00000	50.000000	0.000000	0.000000	55.000000
25%	16.000000	70.000000	6.00000	63.000000	1.000000	2.000000	65.000000
50%	20.000000	80.000000	7.00000	75.000000	1.000000	3.000000	67.000000
75%	24.000000	90.000000	8.00000	88.000000	2.000000	4.000000	69.000000
max	44.000000	100.000000	10.00000	100.000000	8.000000	6.000000	101.000000

```
df.isnull().sum()
```

	0
Hours_Studied	0
Attendance	0
Parental_Involvement	0
Access_to_Resources	0
Extracurricular_Activities	0
Sleep_Hours	0
Previous_Scores	0
Motivation_Level	0
Internet_Access	0
Tutoring_Sessions	0
Family_Income	0
Teacher_Quality	78
School_Type	0
Peer_Influence	0
Physical_Activity	0
Learning_Disabilities	0
Parental_Education_Level	90
Distance_from_Home	67
Gender	0
Exam_Score	0

dtype: int64

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

df['Teacher_Quality'] = le.fit_transform(df['Teacher_Quality'])
df['Parental_Education_Level'] = le.fit_transform(df['Parental_Education_Level'])
df['Distance_from_Home'] = le.fit_transform(df['Distance_from_Home'])
df
```

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Previous_Sc
0	23	84	Low	High	No	7	
1	19	64	Low	Medium	No	8	
2	24	98	Medium	Medium	Yes	7	
3	29	89	Low	Medium	Yes	8	
4	19	92	Medium	Medium	Yes	6	
...	...	...	...	...	...	...	
6602	25	69	High	Medium	No	7	
6603	23	76	High	Medium	No	8	
6604	20	90	Medium	Low	Yes	6	
6605	10	86	High	High	Yes	6	
6606	15	67	Medium	Low	Yes	9	

6607 rows × 20 columns

```
df.isnull().sum()
```

	0
Hours_Studied	0
Attendance	0
Parental_Involvement	0
Access_to_Resources	0
Extracurricular_Activities	0
Sleep_Hours	0
Previous_Scores	0
Motivation_Level	0
Internet_Access	0
Tutoring_Sessions	0
Family_Income	0
Teacher_Quality	0
School_Type	0
Peer_Influence	0
Physical_Activity	0
Learning_Disabilities	0
Parental_Education_Level	0
Distance_from_Home	0
Gender	0
Exam_Score	0

dtype: int64

```
le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
```

```
df
```

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Previous_Sc
0	23	84	1	0	0	7	
1	19	64	1	2	0	8	
2	24	98	2	2	1	7	
3	29	89	1	2	1	8	
4	19	92	2	2	1	6	
...	...	...	...	...	...	...	
6602	25	69	0	2	0	7	
6603	23	76	0	2	0	8	
6604	20	90	2	1	1	6	
6605	10	86	0	0	1	6	
6606	15	67	2	1	1	9	

6607 rows × 20 columns

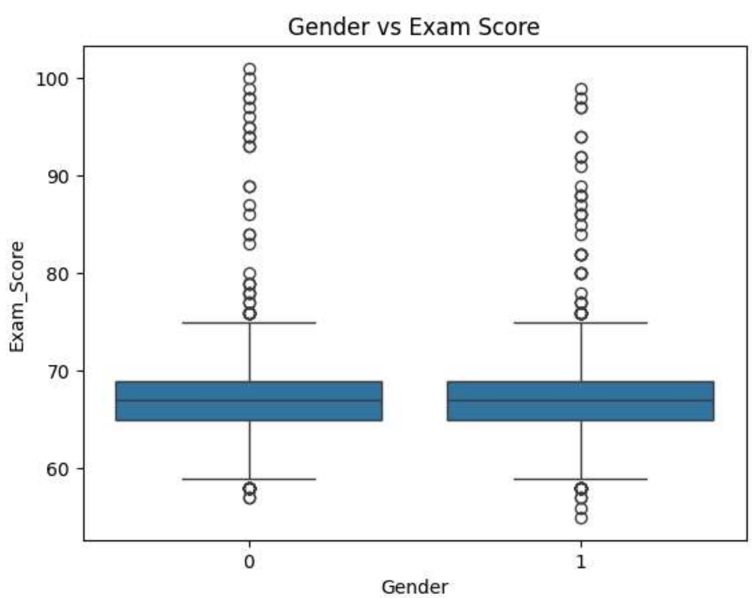
```
# Gender vs Exam Score
print(pd.crosstab(df['Gender'], df['Exam_Score']))
```

```
Exam_Score  55   56   57   58   59   60   61   62   63   64   ...  92   93   \
Gender                                                        ...
0            0    0    2   11   17   38   70  105  163  223  ...    0    2
1            1    1    2   11   23   39  101  159  208  278  ...    2    0

Exam_Score  94   95   96   97   98   99   100  101
Gender
0            2    2    1    1    2    1    1    1
1            2    0    0    2    1    1    0    0

[2 rows x 45 columns]
```

```
sns.boxplot(x='Gender', y='Exam_Score', data=df)
plt.title("Gender vs Exam Score")
plt.show()
```




```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.ensemble import RandomForestRegressor
```

```
X = df.drop("Exam_Score", axis=1)
y = df["Exam_Score"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = RandomForestRegressor(n_estimators=100, random_state=42)

model.fit(X_train, y_train)
```

```
▾ RandomForestRegressor
RandomForestRegressor(random_state=42)
```

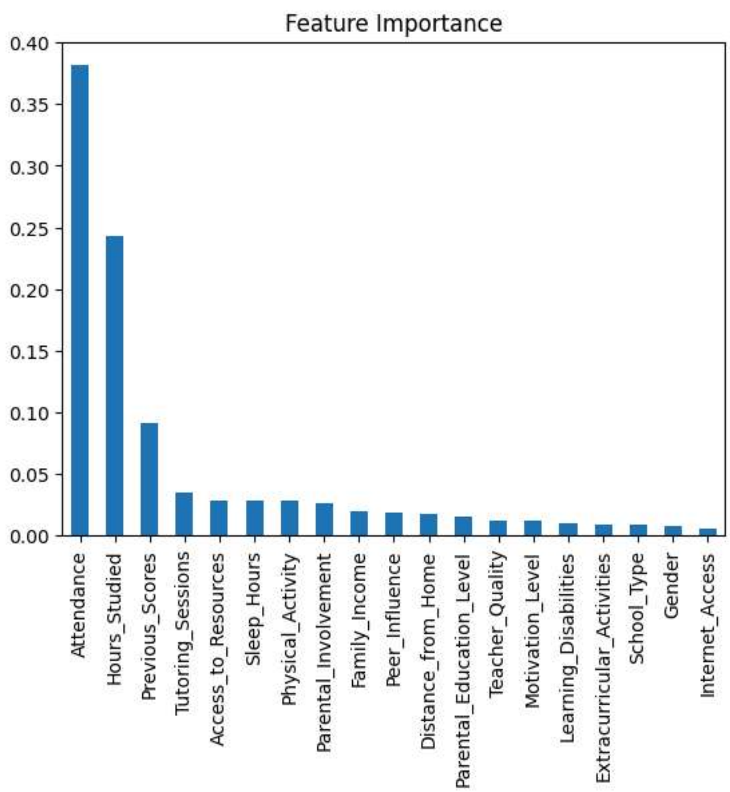
```
y_pred = model.predict(X_test)
```

```
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
MAE: 1.1328819969742814
R2 Score: 0.6532863038564358
```

```
importance = pd.Series(model.feature_importances_, index=X.columns)

importance.sort_values(ascending=False).plot(kind='bar')
plt.title("Feature Importance")
plt.show()
```




```
X_fair = df.drop(["Exam_Score", "Gender"], axis=1)
```

```
X_train, X_test, y_train, y_test = train_test_split(X_fair, y, test_size=0.2, random_state=42)
```

```
model2 = RandomForestRegressor()
model2.fit(X_train, y_train)
```

```
▾ RandomForestRegressor
RandomForestRegressor()
```

```
pred = model2.predict(X_test)
```

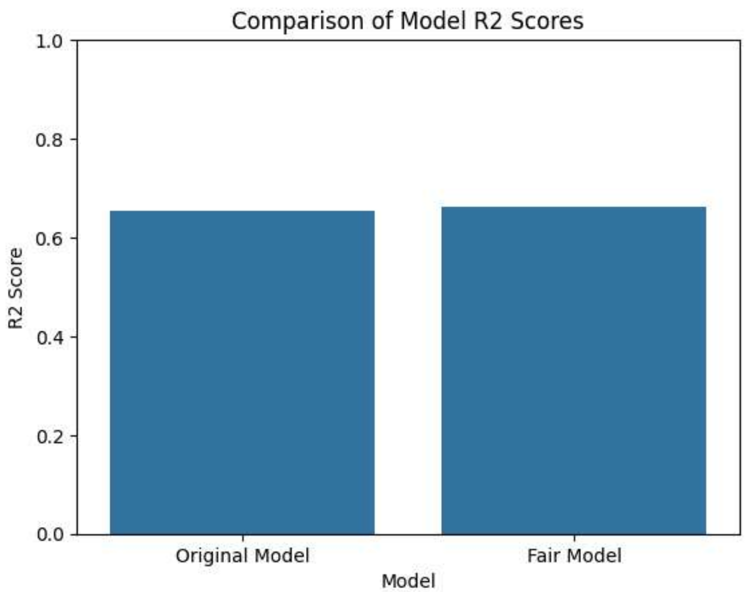
```
print("Fair Model R2:", r2_score(y_test, pred))
```

```
Fair Model R2: 0.6609432806096592
```

```
r2_model1 = r2_score(y_test, y_pred)
r2_model2 = r2_score(y_test, pred)

model_scores = pd.DataFrame({
    'Model': ['Original Model', 'Fair Model'],
    'R2 Score': [r2_model1, r2_model2]
})

sns.barplot(x='Model', y='R2 Score', data=model_scores)
plt.title('Comparison of Model R2 Scores')
plt.ylabel('R2 Score')
plt.ylim(0, 1)
plt.show()
```




```
Start coding or generate with AI.
```